



**databricks**

# Genie Code Workshop Labs

## Table of Contents

### Introdução

O que você vai aprender

Trabalhando com Free Edition e Genie Code

Recursos

### Setup

Lab 1: Primeiros Passos — Setup, Modos e Navegação

Lab 2: Análise Exploratória de Dados com Agent Mode

Lab 3: Otimização e Debug de Queries SQL

Lab 4: Workflow de Dados com Lakeflow Designer

Lab 5a: Workflow de Dados com Lakeflow Spark Declarative Pipelines

Lab 5b: Workflow de Dados com Lakeflow Spark Declarative Pipelines (Auto-Dirigido)

Lab 6a: Análise e Visualização de Dados

Lab 6b: Análise e Visualização de Dados (Auto-Dirigido)

Lab 7a: Predição de Preços com Machine Learning

Lab 7b: Predição de Preços com Machine Learning (Auto-Dirigido)

Lab 8: Analisando Texto com Funções AI SQL

Lab 9: Genie Spaces — Converse com seus Dados

Lab 10: Personalizar o Genie Code

### E Agora?

Continue Praticando no Seu Próprio Workspace

Aprofunde-se nos Tópicos deste Lab

Assista e Aprenda

Comunidade e Eventos

---

# Introdução

O [Genie Code](#) é um parceiro autônomo de IA projetado para trabalho com dados no Databricks. Diferente de outros assistentes de IA, o Genie Code é profundamente integrado ao Unity Catalog, permitindo que ele compreenda todo o seu cenário de dados, incluindo tabelas, colunas e linhagem. Essa consciência contextual permite ao Genie Code acelerar workflows complexos de dados enquanto se adapta autonomamente ao seu modelo de dados e governança específicos. O Genie Code foi projetado para equipes de dados usarem diariamente, desde experimentação e desenvolvimento de modelos até pipelines de produção e dashboards de BI. Ele está disponível em todo o Databricks Lakehouse, e os threads de chat persistem enquanto você navega entre páginas.

Estes labs práticos vão guiá-lo pelas capacidades principais do Genie Code usando um workspace [Databricks Free Edition](#).

## O que você vai aprender

- **Lab 1: Primeiros Passos - Setup, Modos e Navegação** - Familiarize-se com o básico do Genie Code.
- **Lab 2: Análise Exploratória de Dados com Agent Mode** - Deixe o Agent Mode planejar e executar autonomamente uma análise exploratória completa (EDA).
- **Lab 3: Otimização e Debug de Queries SQL** - Formate, documente, otimize e corrija SQL quebrado.
- **Lab 4: Lakeflow Designer** - Construa um pipeline ETL simples no-code usando o canvas do Lakeflow Designer e prompts.
- **Lab 5a e 5b: Construir um Workflow de Transformação de Dados** - Construa um pipeline ETL medallion completo com Lakeflow Spark Declarative Pipelines, estendendo-o iterativamente via prompts de follow-up.
- **Lab 6a e 6b: Análise e Visualização de Dados** - Crie um Dashboard AI/BI a partir das tabelas do Lab 5a ou 5b - tudo dirigido por linguagem natural.
- **Lab 7a e 7b: Predição de Preços com Machine Learning** - Construa, treine e avalie um modelo de ML end-to-end que preveja preços de imóveis.
- **Lab 8: Funções AI SQL** - Use várias funções de IA no Databricks como `ai_classify`, `ai_summarize` e `ai_query`.
- **Lab 9: Genie Spaces** - Construa um Genie Space para suportar análise em linguagem natural.
- **Lab 10: Personalizar o Genie Code** - Direcione o Genie Code para trabalhar do seu jeito.

## IMPORTANTE:

- Você pode completar os labs sem experiência prévia com Databricks, mas um entendimento básico de conceitos como Notebooks, Spark Declarative Pipelines, Machine Learning, etc. ajudará você a apreciar melhor o valor do Genie Code.

- Além do lab de Setup, os labs 1–5, 8–10 são independentes sem pré-requisitos entre si. Os Labs 6 e 7 dependem das tabelas criadas no Lab 5.
- É recomendado completar os Labs 1–3 antes de tentar qualquer outro Lab, pois isso o familiarizará com o uso do Genie Code.
- Labs Auto-Dirigidos (5b, 6b, 7b): cobrem o mesmo terreno dos Labs 5a, 6a e 7a — mas em vez de fornecer os prompts, dão um ponto de partida e um objetivo, e deixam você conduzir a conversa com o Genie Code. São destinados a participantes confortáveis o suficiente com Databricks e Genie Code para desenhar o workflow por conta própria. Descreva resultados, não passos. Diga ao Genie Code o que você quer, não como construir. Itere via follow-ups quando a primeira tentativa não estiver perfeita.
- As instruções nos labs iniciais são mais detalhadas do que nos labs posteriores — a premissa é que você não precisa de cada detalhe específico conforme avança. Por exemplo, no início da maioria dos labs há um checklist de passos que você deve completar antes de prosseguir (**similar** ao seguinte). **Pular qualquer um deles quase certamente causará problemas.** Completar o Setup ajudará você a entender como completar esses passos.
- Você não precisa completar todos os labs de uma só vez. Foque nos mais relevantes para seu papel. Lembre-se que esses labs rodam na Free Edition — fiel ao nome, é gratuita — então você pode progredir no seu ritmo.

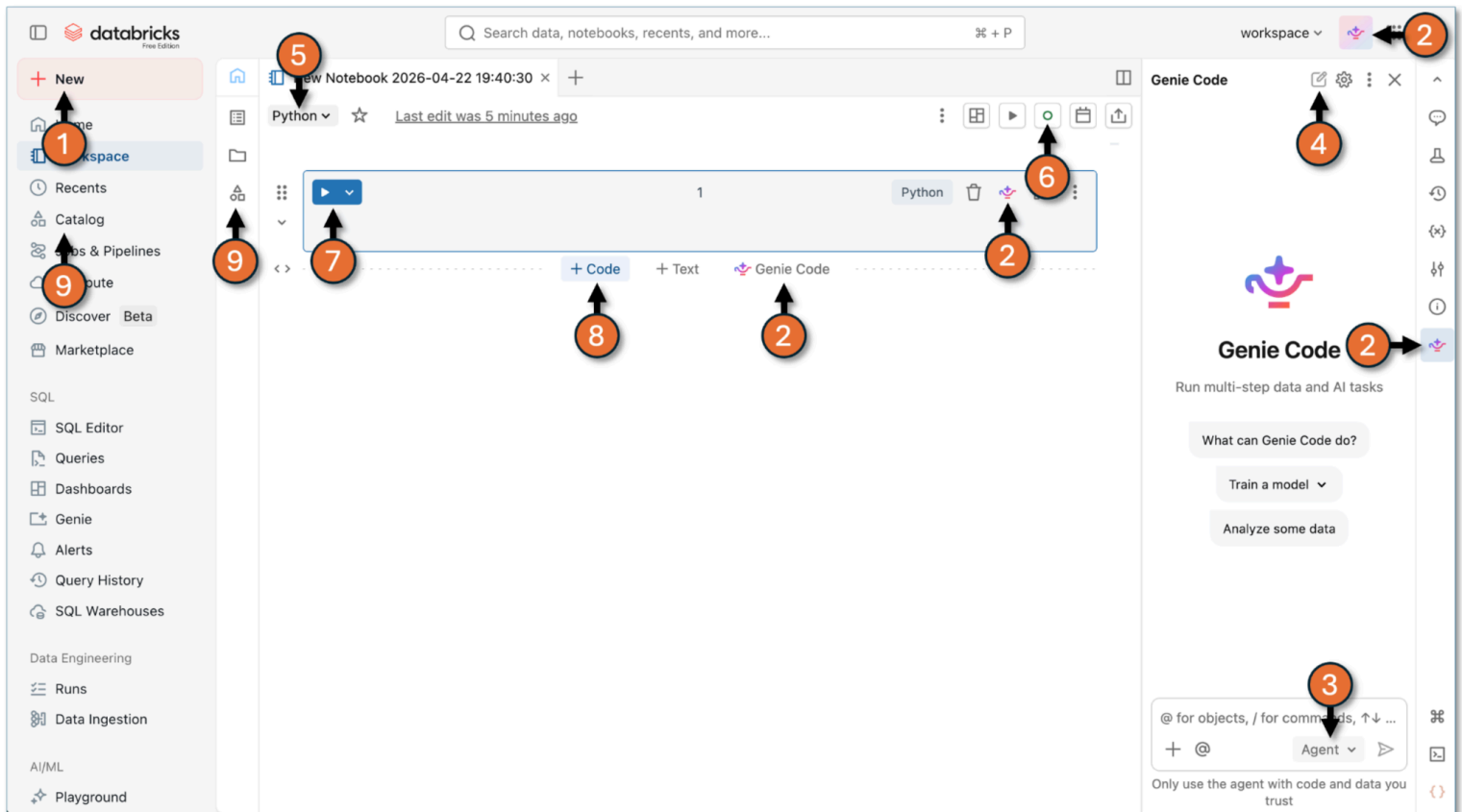
## Trabalhando com Free Edition e Genie Code

- **A Free Edition compartilha recursos de back-end** e tem cotas diárias de compute serverless e [outras limitações](#), então espere que seja mais lenta que o Databricks de produção. Mas é gratuita!
- **O Genie Code é não-determinístico** — seus resultados podem variar. Então:
  - Sempre **revise o código gerado** antes de executar — o Genie Code é poderoso mas pode cometer erros.
  - **Seja específico** — mais contexto = melhor resultado. Inclua nomes de colunas, formatos esperados e tipos de visualização desejados.
  - **Experimente ambos os modos** — alterne entre Agent Mode (para executar) e Chat Mode (para entender) para ver como diferem.
  - **Trabalhe em um chat por lab** — o contexto se acumula, e o Genie Code fica melhor em prever o que você quer conforme a conversa cresce.
  - **Se o Genie Code sair dos trilhos** — pare-o (botão quadrado vermelho) e re-prompt em vez de deixá-lo construir sobre uma base ruim.
  - **O mais importante: brinque** — escreva seus próprios prompts para explorar. Se encontrar erros, trabalhe com o Genie Code para resolver.

- Use **@<nome-da-tabela>** para apontar o Genie Code para uma tabela específica. Quando colar um nome completo deste documento (ex: @samples.tpch.orders), clique no texto colado e selecione a tabela do seletor. Atalho: digite apenas @orders e escolha a correta da lista.
- **Aprovações e controles** — O Genie Code pede aprovação antes de executar código ou atualizar pipelines. Clique **'Allow'**, **'Decline'**, **'Continue'** ou **'Reject'** conforme apropriado. Você também pode selecionar **'Allow in this thread'** ou **'Always allow'** para reduzir prompts. Nestes labs, recomendamos clicar **'Allow'** (um passo de cada vez) para que você possa observar o que está acontecendo. Para parar o Genie Code no meio de uma tarefa, clique o botão vermelho de stop.
- O Genie Code é uma tecnologia em **rápida evolução**. A experiência e funcionalidades podem mudar (para melhor) quando você tentar esses labs novamente.

## Recursos

- [Documentação Databricks Free Edition](#)
  - Documentação Genie Code ([AWS](#), [Azure](#), [GCP](#))
  - [Introducing Genie Code](#) — anúncio de lançamento e visão geral de funcionalidades
  - [Get to Know Genie Code](#) — demo cobrindo pipelines, dashboards e Agent Mode
  - [Canal YouTube Databricks](#) (filtrado por 'Genie Code') — tutoriais e demos
  - [Databricks Genie Code: Full Practice Guide and Examples](#)
  - [From Genie to Genie Code: Agentic Data Work on Databricks](#)
  - [Agentic Data Engineering with Genie Code and Lakeflow](#)
-



1. Clique para criar um novo Notebook / Pipeline ETL / etc.
2. Abrir o painel do Genie Code
3. Alternar o Genie Code entre os modos Chat e Agent (Agente)
4. Iniciar um novo chat
5. Alternar a linguagem padrão do Notebook entre Python e SQL
6. Selecionar o tipo de computação (*Serverless* ou *Serverless Starter Warehouse*)
7. Executar célula
8. Clique em '+ Code' para adicionar uma célula vazia abaixo da célula atual
9. Unity Catalog - explorar objetos relacionados a dados

## Setup

**Objetivo:** Configurar seu workspace Databricks Free Edition; Criar um schema onde objetos como tabelas serão criados.

1. Se ainda não fez, registre-se para o **Databricks Free Edition** em [databricks.com/try-databricks](https://databricks.com/try-databricks) (e-mail corporativo ou pessoal).
2. Uma vez conectado ao seu workspace Databricks Free Edition, clique em **'+ New'** no menu da esquerda, depois selecione **'Notebook'**.
3. No topo esquerdo você pode ver **'Python'**. Clique nele e mude para **'SQL'** (isso define a linguagem padrão do notebook).
4. Abra o painel Genie Code clicando no ícone do **Genie Code** no canto superior direito.
5. Garanta que **'Agent'** (vs **'Chat'**) está selecionado no canto inferior direito.
6. No painel do chat, digite o seguinte prompt:

*Crie um novo schema workspace.genie\_demo.*

7. Quando solicitado, clique **'Allow'** para que o Genie Code possa executar o código gerado e criar o schema que será usado por vários labs.

## Lab 1: Primeiros Passos — Setup, Modos e Navegação

**Objetivo:** Configurar seu workspace Databricks Free Edition; Aprender a interagir com o Genie Code; Explorar a UI; Usar referências @ de tabelas.

1. **'+ New' → 'Notebook'**. Linguagem padrão **'SQL'**. Painel Genie Code aberto. **'New Chat'**. Modo **Agent**.
2. No painel do chat, digite o seguinte prompt:

*Quais tabelas estão disponíveis no schema samples.tpch? Descreva cada tabela e como elas se relacionam entre si.*

3. O Genie pode pedir para executar código, potencialmente várias vezes. Durante estes labs recomendamos aceitar o padrão **'Ask every time'** e clicar **'Allow'**.
4. Revise a resposta final. O Genie Code usa metadados do Unity Catalog para descrever as tabelas e seus relacionamentos.
5. Use a referência @ para fazer uma pergunta específica sobre uma tabela:

*Quantas colunas a tabela @samples.tpch.orders tem? O que cada coluna representa?*

6. Agora, peça ao Genie Code para gerar uma query simples:

*Escreva uma query SQL que mostre os 10 clientes com maior valor total de pedidos usando @samples.tpch.orders e @samples.tpch.customer.*

7. O Genie Code deve gerar o SQL em uma nova célula no notebook. Quando solicitado para executar a célula, clique **'Allow'**. Você também pode ver botões **'Run suggested'**, **'Reject'** e **'Accept'** na célula — ignore-os e direcione a execução via **'Allow'** do Genie Code.
8. Experimente o comando `/explain`. Pode ser feito de 3 formas:
  - clique no kebab no topo direito da célula e clique no ícone do Genie e digite `/explain`
  - na célula, pressione **'Cmd+I'** (Mac) ou **'Ctrl+I'** (Windows) e digite `/explain`
  - no painel do Genie Code digite: explique esta query linha por linha
9. Note que existem outros [/commands](#) que você pode usar com o Genie Code incluindo (mas não limitado a):
  - `/findTables` — busca tabelas relevantes baseado em metadados do Unity Catalog
  - `/getTableLineages` — visualiza dependências upstream e downstream
  - `/getTableInsights` — acessa insights baseados em metadados, como atividade de usuários e padrões de queries
10. Mude o Genie Code para o modo **'Chat'** (canto inferior direito do painel do Genie Code, onde deve estar mostrando **'Agent'**). Digite a seguinte pergunta:

*O que é uma window function no Spark? Como ela difere de um GROUP BY normal? Quando eu usaria uma ao invés da outra?*

11. Faça o follow up com:
 

*Qual é a diferença entre RANK(), DENSE\_RANK() e ROW\_NUMBER()? Mostre um exemplo usando samples.tpch.orders onde a diferença importa — onde as três funções produzem resultados diferentes.*
12. Se um código de exemplo aparecer no painel do Genie Code, use o botão **'<<'** no topo do snippet para inseri-lo em uma nova célula. Para criar uma nova célula, mova o mouse abaixo da última célula até **'- - +Code +Text Genie Code - - '** aparecer — clique em **'+Code'**. Uma vez que o código estiver inserido na nova célula, execute a instrução SQL (clique o botão de executar no topo esquerdo da célula). Verifique onde as três funções divergem.

## Lab 2: Análise Exploratória de Dados com Agent Mode

**Objetivo:** Usar o Genie Code para realizar uma análise exploratória de dados completa em um dataset, demonstrando a capacidade do Genie Code de planejar, executar e iterar autonomamente.

1. **'+' New** → **'Notebook'**. Linguagem padrão **'Python'**. Painel Genie Code aberto. **'New Chat'**. Modo **Agent**.
2. Digite o seguinte prompt:

*Realize uma análise exploratória de dados na tabela @samples.nyctaxi.trips. Inclua: contagem de linhas, visão geral do schema, estatísticas descritivas para colunas numéricas, distribuição das distâncias de viagem, tendências de valor da corrida por hora do dia, e identifique quaisquer problemas de qualidade de dados. Use Python com matplotlib para as visualizações.*

3. Observe como o Genie Code planeja os passos da análise / cria múltiplas células no notebook.
4. Uma vez que as células foram criadas, execute cada célula individualmente no Notebook (em oposição ao Genie Code) ou direcione o Genie Code para executar as células usando **run suggested** no topo esquerdo de cada célula. Para qualquer célula que você queira entender melhor, selecione o código, pressione **'Cmd+I'** (Mac) ou **'Ctrl+I'** (Windows), e digite: `/explain`
5. Se encontrar erros, siga os prompts para usar o Genie para diagnosticar o erro e reexecutar código revisado (pode ser mais fácil clicar **'Accept'** no Genie Code e deixá-lo executar as células novamente — ele deve tratar erros via retry, repetindo se necessário até funcionar).
6. Se ainda não o fez, no painel do Genie Code clique **Allow** para executar todas as células. Ele vai reexecutar as mesmas células, o que é ok, antes de avançar para resumir suas descobertas.
7. Agora faça uma pergunta de follow-up no painel do Genie Code:
 

*Com base na EDA, quais são os padrões mais interessantes? Existem outliers que valem a pena investigar?*
8. Deixe o Agent Mode aprofundar nos padrões identificados (o que provavelmente incluirá células de código adicionais — execute-as via o painel do Genie Code quando solicitado).
9. Revise a análise mais aprofundada.

## Lab 3: Otimização e Debug de Queries SQL

**Objetivo:** Usar os slash commands do Genie Code para organizar, otimizar e diagnosticar erros em queries SQL.

**Novidade neste lab:** O Genie Code inclui três comandos voltados para higiene e performance de SQL: `/prettify` (formatação), `/doc` (comentários inline) e `/optimize` (reescrita para performance). Quando uma célula tem erros, o recurso Diagnose Error do Genie Code lê a falha, propõe uma correção e oferece reexecutar. Alguns comandos rodam no painel do Genie Code e alguns no assistente inline (Cmd+I / Ctrl+I) — o lab aponta qual é qual.



1. **'+' New** → **'Notebook'**. Linguagem padrão **'SQL'**. Pannel Genie Code aberto. **'New Chat'**. Modo **Agent**.
2. Cole a seguinte instrução SQL na célula vazia no topo do Notebook — ela é intencionalmente subótima e desordenada:

```
SELECT c.c_name, sum(l.l_extendedprice * (1 - l.l_discount)) as revenue
FROM samples.tpch.customer c, samples.tpch.orders o, samples.tpch.lineitem
1
WHERE c.c_custkey = o.o_custkey and o.o_orderkey = l.l_orderkey
AND o.o_orderdate >= '1994-01-01' and o.o_orderdate < '1995-01-01'
GROUP BY c.c_name ORDER BY revenue DESC
```

1. Usando o painel do Genie Code, digite: `/prettify`
2. Você deve ver a versão original destacada em vermelho e as mudanças recomendadas destacadas em verde. Clique **'Accept'** para aceitar a versão formatada. A query deve ficar mais legível.
3. Este próximo passo não acontece no painel do Genie Code! Em vez disso, destaque o código SQL na célula, e clique no kebab no topo direito da célula e clique no ícone do Genie ou pressione **'Cmd+I'** (Mac) ou **'Ctrl+I'** (Windows), e digite: `/doc`
4. **'Accept'** a documentação — o Genie Code adicionará comentários inline claros explicando a lógica da query.
5. Execute a célula (clique o botão de executar no topo esquerdo da célula).
6. De volta no painel do Genie Code, digite:

*A query está subótima? Se sim, por quê?*

1. Peça ao Genie Code para otimizá-la: `/optimize`
2. Revise as mudanças sugeridas — o Genie Code pode sugerir substituir joins implícitos por sintaxe JOIN explícita, adicionar filtros apropriados ou reestruturar a query.
3. Aceite e execute o código ajustado via o painel do Genie Code — uma vez feito, você deve receber uma explicação de porque esta nova versão é uma melhoria em relação à versão anterior.
4. Um botão **'Optimize'** também aparece no canto inferior direito da célula SQL. Opcionalmente clique nele para uma análise pós-execução, e aplique quaisquer recomendações adicionais.
5. Se você executou a query antes e depois de `/optimize`, tente **'compare execution times'** (oferecido como um botão, ou digite no Genie Code se não vir esse botão).
6. Agora, crie uma nova célula SQL com a seguinte query SQL quebrada (mova o mouse abaixo da última célula até **'- - +Code +Text Genie Code - -'** aparecer — clique em **'+Code'**):

```
SELECT c_name, total_spent
FROM samples.tpch.customer
JOIN (
  SELECT o_custkey, SUM(o_totalprice) AS total_spent
  FROM samples.tpch.orders
```

```
WHERE o_orderdate > '1998-01-01'
GROUPBY o_custkey
) AS order_totals
ON c_custkey = order_totals.o_custkey
```

1. Execute a célula — ela vai falhar.
2. O Genie Code pode ir direto para **'Attempting to fix'**. Se não, clique o botão **'Diagnose Error'** que aparece no output da célula.
3. O Genie Code identificará os problemas (ex: GROUPBY deveria ser GROUP BY, possíveis problemas de range de datas) e sugerirá uma correção rápida. Clique **'Allow'** para aplicar e reexecutar.

## Lab 4: Workflow de Dados com Lakeflow Designer

**Objetivo:** Construir um pipeline ETL no-code no Lakeflow Designer que faz upload de um CSV, aplica transformações de qualidade (normalização de maiúsculas/minúsculas + filtro de linhas), junta com uma segunda fonte do Unity Catalog, agrega e grava uma tabela resultado — tudo dirigido pelo Genie Code a partir do canvas do Designer.

**Novidade neste lab:** O Lakeflow Designer é um construtor visual de pipelines no-code para ETL no Databricks. Ele usa um canvas drag-and-drop mais Genie Code, para que um analista de negócios possa construir um pipeline de produção sem escrever código. Por trás dos bastidores, cada pipeline do Designer compila para um Lakeflow Spark Declarative Pipeline (veja Labs 5a e 5b) — então ele recebe a mesma governança (Unity Catalog), observabilidade, agendamento e confiabilidade de um pipeline codificado manualmente. Veja [Announcing Lakeflow Designer](#) / [Lakeflow Pipelines Editor](#).

1. **'+ New' → 'Notebook'**. Linguagem padrão **'SQL'**.
2. Cole a seguinte instrução SQL na célula vazia no topo do Notebook:

```
SELECT *
FROM samples.tpch.orders
WHERE o_orderdate >= DATE '1998-07-01'
LIMIT 1000;
```

1. Na tabela de resultados, clique no ícone de download no canto inferior esquerdo da grade de resultados e escolha **'Download CSV'**. Salve o arquivo como **'orders\_recent.csv'** em algum lugar fácil de encontrar.
2. Clique **'+ New' → 'Visual data prep'**. O canvas do Lakeflow Designer abre.
3. Clique **'Upload a file'**, clique em **'browse'**, **'select files'** e selecione **'orders\_recent.csv'** do seu computador.
4. Quando solicitado para um Volume de destino, clique **'+ Create volume'**:
  - Volume name = **'uploads'**

- Volume type = **'Managed volume'**
  - Catalog = **'workspace'**
  - Schema = **'genie\_demo'**
1. Clique **'Create'** e depois **'Upload'**. Se o processamento não passar do "Preview" após um minuto, atualize a aba do navegador.
  2. Uma vez que o upload terminar, um novo nó Source aparece no canvas, conectado ao arquivo **'orders\_recent.csv'** (que agora está armazenado no volume que você criou).
  3. Na seção Preview abaixo, examine as colunas e observe:
    - **'o\_orderpriority'** contém valores como 1-URGENT, 2-HIGH, etc. — já em maiúscula neste dataset, mas demonstraremos a transformação de normalização de qualquer forma.
    - **'o\_totalprice'** varia entre pedidos pequenos e grandes. Vamos filtrar pedidos abaixo de 50.000.
  1. Nos passos seguintes você usará o Genie Code para fazer o trabalho. Note, porém, que você tem a opção de fazer tudo manualmente arrastando operadores do menu da esquerda para o canvas do Designer e/ou clicando com o botão direito no canvas e selecionando o operador desejado.
  2. Além disso, diferente dos labs anteriores onde você usou o painel do Genie Code a partir do topo direito da página, desta vez você tem a opção de usar o painel do Genie Code na parte inferior do canvas do Designer.
  3. No painel do Genie Code no canvas do Designer, digite o seguinte prompt:
 

*Remova as linhas onde o\_totalprice é menor que 50000 e converta a coluna o\_orderpriority para minúsculas.*
  1. O Genie Code deve criar dois novos operadores conectados downstream da fonte. Clique **'Accept All'**.
  2. Inspeção o nó de filtro:
    - Clique no nó do filtro, depois no ícone Edit do operador. Isso abrirá o painel de edição na direita onde você pode ver uma expressão aplicada aos dados fonte similar a `o_totalprice is greater than or equal to "50000"`.
    - Ordene a prévia de output no canto inferior direito por `o_totalprice` ascendente e confirme que o menor valor agora é `>= 50.000`. A contagem de linhas (no fundo) também deve cair das 1000 linhas mostradas no Input preview.
  1. Inspeção o nó de lowercase:
    - Clique no nó, depois no ícone Edit do operador — você deve ver algo como `LOWER(o_orderpriority)`.
    - Compare `o_orderpriority` nas prévias Input / Output para confirmar que a coluna agora está em minúsculas.
  1. Adicione a dimensão de clientes como segunda fonte. Usando o Genie Code, prompt:
 

*Adicione samples.tpch.customer como uma fonte.*
  1. Clique **'Accept All'**.

2. Clique no operador source de Customers, depois mude **'Rows scanned: Limit'** para **'Rows scanned: Max'** no painel de output (veja na parte mais inferior do painel de output). Isso garante que todas as linhas da tabela de clientes sejam trazidas na prévia.
3. Peça ao Genie Code para juntar os dois datasets:

*Junte a fonte de pedidos com a fonte de clientes. Cada pedido deve mostrar os detalhes do cliente.*

1. Clique **'Accept All'**.
2. Organize o canvas: clique com o botão direito no canvas e clique no botão **Auto layout** ou selecione-o do topo do canvas.
3. Clique no nó do Join, depois no ícone Edit do operador. Note:
  - As chaves de join (`o_custkey = c_custkey`).
  - O tipo de join (`Left join`).
  - As linhas produzidas no output incluem tanto colunas de fato quanto de dimensão (ex: `o_totalprice` e `c_name`).

1. Peça ao Genie Code para agregar:

*Me dê a média de `o_totalprice` e a contagem de pedidos, agrupados por segmento de mercado do cliente (`c_mktsegment`).*

1. Clique **'Accept All'**.
2. Inspeção o novo nó de agregação — Logic e Input/Output preview devem mostrar uma linha por segmento de mercado com o preço médio e contagem de pedidos.
3. Peça ao Genie Code para gravar os resultados em uma tabela:

*Grave os dados em uma tabela chamada `orders_avg_by_segment` no schema `workspace.genie_demo`.*

1. Clique **'Accept All'**.
2. Clique no nó final de output, depois no ícone Edit do operador — você deve ver **'Run'** no editor do operador.
3. Clique **'Run'**.
4. Uma vez que a execução terminar com sucesso, clique em **'View in Catalog'** no editor do operador (símbolo de tabela no topo direito). Isso abrirá o Unity Catalog em uma nova aba.
5. Clique **'Sample Data'** (pode precisar iniciar o compute). Você deve ver cinco linhas — uma por segmento de mercado TPC-H (AUTOMOBILE, BUILDING, FURNITURE, HOUSEHOLD, MACHINERY) com preço médio e contagem de pedidos.
6. Clique **'Lineage'**, **'See lineage graph'** para ver o notebook Python associado (que sempre abre no Lakeflow Designer) e as fontes (arquivo `orders_recent.csv` no volume e a tabela de clientes no schema `samples.tpch`).

## Lab 5a: Workflow de Dados com Lakeflow Spark Declarative Pipelines

**Objetivo:** Usar o Genie Code para construir um workflow simples de transformação de dados usando Lakeflow Spark Declarative Pipelines, demonstrando as capacidades de data engineering do Genie Code. **Este lab contém todos os prompts necessários para atingir o objetivo. Se preferir DIY, pule para o Lab 5b.**

**Novidade neste lab:** Lakeflow Spark Declarative Pipeline é uma forma declarativa, gerenciada pelo framework, de construir pipelines ETL batch ou streaming em SQL ou Python. Você descreve as tabelas-alvo que deseja; o Databricks resolve as dependências, ordenação, retries e processamento incremental para você. Pipelines são governados pelo Unity Catalog e executam no Databricks Runtime. Veja [Lakeflow Spark Declarative Pipelines](#) / [Concepts](#) / [Medallion architecture](#).

1. **'+ New' → 'ETL pipeline'**. Linguagem padrão **'SQL'**. Painel Genie Code aberto. **'New Chat'**. Modo **Agent**.
2. Na UI do ETL Pipeline, mude o catálogo e schema do pipeline de **'workspace.default'** para **'workspace.genie\_demo'** (topo esquerdo) para evitar problemas de permissão de catálogo.
3. Digite o seguinte prompt para criar as tabelas bronze:

*Crie um pipeline com streaming tables bronze que ingiram as tabelas samples.wanderbricks: properties, reviews, hosts, amenities, property\_amenities, destinations. Sem transformações, apenas adicione comentários nas tabelas. Schema alvo: workspace.genie\_demo.*

1. O Genie Code pode ir em frente e criar os arquivos Python necessários. Ele pode em vez disso fornecer um plano (ex: você pode ver 'Does this structure work for you, or would you like to modify the naming or organization?'), seguido de alguns prompts de follow-up sugeridos. Clique no prompt que cria os arquivos Python (ex: 'Yes, proceed with this structure').
2. Sinta-se à vontade para clicar em qualquer um dos arquivos gerados para ver o código Python gerado para cada tabela.
3. Quando solicitado **'Dry-running pipeline'**, clique **'Allow'**. Você também pode ver um aviso **"blocked execution for safety reasons"** — clique **'run'** para continuar.
4. O dry-run verifica o código para garantir que funciona como uma unidade. Se o dry run falhar, ele deve diagnosticar o que deu errado e tentar novamente.
5. Uma vez que o dry run completar sem erro, você será solicitado a **'Running pipeline'**. Clique **'Skip'**.
6. Agora peça ao Genie Code para criar as tabelas silver:

*Adicione tabelas silver no mesmo schema que limpam a camada bronze: converta preços para double e datas para tipo date, adicione expectations que descartem linhas onde base\_price é null ou zero, junte amenities a property\_amenities para obter os nomes das amenidades por propriedade, agregue reviews por*

*propriedade (média de scores, contagem de reviews, data do review mais recente), padronize categóricos para minúsculas. Calcule host\_tenure\_days a partir da data de início do host.*

1. Uma vez que o dry run completar sem erro, você será solicitado a **'Running pipeline'**. Como antes, clique **'Skip'**.
2. Finalmente, peça ao Genie Code para criar a tabela gold:

*Adicione uma tabela gold chamada gold\_price\_prediction\_features no mesmo schema que junte todas as tabelas silver em uma tabela desnormalizada com uma linha por propriedade. Inclua base\_price, features da propriedade, features do host incluindo host\_tenure\_days, features de destino/localização, métricas agregadas de reviews, e pivote as 15 amenidades mais comuns em colunas de flag binário. Adicione features calculadas: price\_per\_person e price\_per\_bedroom. Adicione comentários nas colunas.*

1. Uma vez que o dry run completar sem erro, quando solicitado **'Running pipeline'**, desta vez clique **'Allow'** (alternativamente clique **'Run Pipeline'** na UI do pipeline) para a criação das tabelas do pipeline.
2. Uma vez que o pipeline começar a executar, mude da view do Genie Code para o Pipeline Graph (topo direito, na extrema direita) e monitore a execução do pipeline (expanda o painel para ver o Pipeline Graph mais claramente).
3. Após o pipeline completar, você pode voltar ao Genie Code para ver as verificações finais e mensagens que o Genie Code aplicou.
4. Clique em **'Runs'** sob **'Data Engineering'** no menu da esquerda.
5. Clique em **'Pipelines'** — você deve ver a execução do seu pipeline. Clique no Start time ou Nome do pipeline para abrir a execução e explorar.
6. No Pipeline graph, passe o mouse sobre gold\_price\_prediction\_features, clique no kebab (3 pontos) → **'View in catalog'**. Na aba do Unity Catalog que abre, clique **'Sample Data'** para ver as linhas (pode precisar clicar **'Select Compute'** primeiro).

## Lab 5b: Workflow de Dados com Lakeflow Spark Declarative Pipelines (Auto-Dirigido)

**Objetivo:** Usar o Genie Code para construir um workflow simples de transformação de dados usando Lakeflow Spark Declarative Pipelines — desenhe e prompt o pipeline você mesmo (consulte o Lab 5a se precisar de ajuda).

1. **'+ New'** → **'ETL pipeline'**. Linguagem padrão **'SQL'**. Painel Genie Code aberto. **'New Chat'**. Modo **Agent**.
2. No topo da UI do pipeline, garanta que o catálogo/schema está definido como workspace.genie\_demo (topo esquerdo).

3. **Ponto de partida:** O schema `samples.wanderbricks` contém tabelas fonte para dados de aluguel de curta duração: `properties`, `reviews`, `hosts`, `amenities`, `property_amenities`, `destinations`.
  4. **Objetivo:** Um Lakeflow Spark Declarative Pipeline produzindo uma única tabela `gold_price_prediction_features`, pronta para alimentar um modelo de ML que preveja `base_price`. A tabela deve ser desnormalizada (uma linha por propriedade), puxar features das seis tabelas fonte (atributos de propriedade, atributos de host, review aggregates, amenity flags), e conter pelo menos duas features calculadas.
  5. **Restrições:**
    - Use o padrão medallion: bronze → silver → gold
    - Bronze deve ser **streaming** sem transformações — apenas ingestão mais comentários de tabela
    - Silver deve aplicar data quality (ex: descartar linhas onde `base_price` é null ou zero, converter preços para double e datas para tipo date)
    - Silver deve agregar reviews por propriedade (média de scores, contagem de reviews, data do review mais recente)
    - Gold deve pivotar as amenidades mais comuns em colunas de flag binário
    - Todas as tabelas devem ir para `workspace.genie_demo`
  1. **Como abordar:**
    - Usando o Genie Code, descreva o pipeline medallion que você quer com suas próprias palavras.
    - Resista à tentação de ditar cada coluna — deixe o Genie Code fazer escolhas sensatas e itere via follow-ups quando o resultado não estiver perfeito.
    - Use referências `@samples.wanderbricks.<table>` quando quiser ser específico.
    - Quando solicitado para dry-run, clique "Allow"; quando solicitado para executar o pipeline completo, "Skip" até ter as três camadas prontas, depois "Allow" a execução final.
  1. **Pronto quando:**
    - O pipeline executar end-to-end sem erro e o Pipeline Graph mostrar o fluxo bronze → silver → gold
    - `gold_price_prediction_features` é consultável e junta dados de pelo menos quatro tabelas fonte
    - Você consegue explicar (e o Genie Code consegue explicar) cada transformação no pipeline
- 

## Lab 6a: Análise e Visualização de Dados

**Objetivo:** Usar o Genie Code dentro de AI/BI Dashboards para construir um dashboard de negócios interativo a partir das tabelas do pipeline criado no Lab 5a ou 5b, demonstrando como



o Genie Code pode gerar datasets e widgets diretamente em um canvas de dashboard. **Este lab contém todos os prompts necessários. Se preferir DIY, pule para o Lab 6b.**

1. **\*\*\* IMPORTANTE \*\*\*** este lab requer que o pipeline do Lab 5a ou 5b tenha completado com sucesso.
2. **'+ New' → 'Dashboard'** (um Dashboard AI/BI em branco abrirá). Painei Genie Code aberto. **'New Chat'**. Modo **Agent**.
3. Digite o seguinte prompt para criar o primeiro dataset e widget (clique **'Allow'** quando solicitado, e em cada vez ao longo deste lab):

*Usando workspace.genie\_demo.gold\_price\_prediction\_features, adicione um dataset e widget mostrando total de propriedades, média de base\_price, média de price\_per\_person e média de price\_per\_bedroom como uma linha de counters no topo do dashboard.*

1. Agora peça ao Genie Code para adicionar um gráfico de distribuição de preços:

*Adicione um dataset e widget mostrando a distribuição de base\_price como um histograma. Título: "Distribuição de Preços".*

1. Em seguida, peça uma análise de features de propriedade:

*Adicione um dataset e widget mostrando a média de base\_price por número de quartos como um gráfico de barras. Título: "Preço Médio por Quartos".*

1. Adicione um widget de métricas de reviews:

*Usando workspace.genie\_demo.silver\_reviews\_aggregated, adicione um dataset e widget mostrando um scatter plot de média de score de review vs contagem total de reviews por propriedade. Título: "Score de Review vs Volume de Reviews".*

1. Adicione uma análise de host, um breakdown por destino e uma análise de impacto de amenidades:

*Usando workspace.genie\_demo.gold\_price\_prediction\_features, adicione um dataset e widget mostrando a relação entre host\_tenure\_days e média de base\_price. Agrupe o tempo de host em intervalos de 6 meses e mostre o preço médio por intervalo como gráfico de barras. Título: "Preço por Experiência do Host". Depois, adicione um dataset e widget mostrando média de base\_price e contagem de propriedades por destino como uma tabela, ordenada por preço médio decrescente. Título: "Resumo por Destino". Finalmente, usando as colunas de flag de amenidades em workspace.genie\_demo.gold\_price\_prediction\_features, adicione um dataset e widget que calcule a média de base\_price para propriedades com vs sem cada uma das top 15 amenidades. Mostre a diferença de preço como um gráfico de barras horizontal ordenado por impacto. Título: "Impacto de Amenidades no Preço".*

1. Finalmente, revise o layout do dashboard. Peça ao Genie Code para organizar:



*Dê um nome à aba do dashboard e reorganize o layout para que os counters fiquem no topo, os gráficos de distribuição de preço e quartos fiquem lado a lado abaixo, depois o scatter plot e o gráfico de experiência do host lado a lado, e a tabela de destinos e o gráfico de impacto de amenidades na parte inferior. Use cores de forma mais eficaz.*

1. Clique na aba **'Data'** (topo esquerdo do Dashboard) para ver todos os datasets que o Genie Code gerou para suportar esses widgets de visualização.
2. Clique **'Publish'** (canto superior direito), aceite os padrões para prompts subsequentes.
3. Clique **'View Published'** (topo central) para ver o Dashboard como os usuários finais o verão (feche o Genie Code).

## Lab 6b: Análise e Visualização de Dados (Auto-Dirigido)

**Objetivo:** Usar o Genie Code dentro de AI/BI Dashboards para construir um dashboard de negócios interativo — desenhe e prompt o Dashboard você mesmo (**consulte o Lab 6a se precisar de ajuda**).

1. **\*\*\* IMPORTANTE \*\*\*** este lab requer que o pipeline do Lab 5a ou 5b tenha completado com sucesso.
2. **'+ New' → 'Dashboard'**. Painel Genie Code aberto. **'New Chat'**. Modo **Agent**.
3. **Ponto de partida:** Um canvas de dashboard em branco mais as tabelas silver e gold em `workspace.genie_demo`.
4. **Objetivo:** Um dashboard publicado que responda à pergunta de negócios *"O que impulsiona o base\_price no nosso portfólio de propriedades, e quais são os mercados mais atrativos para investir?"*
5. **Restrições:** Todos os datasets de `workspace.genie_demo`. Use tanto tabelas gold quanto silver. Misture tipos de widgets. Aplique uma paleta de cores intencional.
6. **Pronto quando:** O dashboard responde à pergunta de negócios e está publicado.

## Lab 7a: Predição de Preços com Machine Learning

**Objetivo:** Usar o Genie Code para construir, treinar e avaliar um modelo de machine learning que preveja o `base_price` de propriedades a partir da tabela de features criada no Lab 5a ou 5b. **Este lab contém todos os prompts necessários. Se preferir DIY, pule para o Lab 7b.**

**Novidade neste lab:** Mosaic AI é a plataforma integrada de ML do Databricks. As duas peças que este lab aborda: **MLflow** (rastreamento automático de experimentos) e **Hyperparameter tuning** (busca de melhores parâmetros).

1. **\*\*\* IMPORTANTE \*\*\*** este lab requer que o pipeline do Lab 5a ou 5b tenha completado com sucesso.
2. **'+ New' → 'Notebook'**. Linguagem padrão **'Python'**. Painel Genie Code aberto. **'New Chat'**. Modo **Agent**.
3. Garanta que o compute anexado é **Serverless**, não **Serverless Starter Warehouse**.
4. Digite o seguinte prompt:

*Usando @workspace.genie\_demo.gold\_price\_prediction\_features, quero prever base\_price usando features da propriedade (bedrooms, bathrooms, accommodates etc), price\_per\_person, price\_per\_bedroom, host\_tenure\_days, métricas de review (média de scores, contagem de reviews), e as colunas de flag de amenidades.*

1. O Genie Code gerará um pipeline de ML multi-célula (EDA, feature engineering, pré-processamento, treinamento). Clique **"Allow"** para executar as células via Genie Code. Para entender qualquer célula, destaque o código e pressione **Cmd+I** (Mac) / **Ctrl+I** (Windows), e digite: /doc ou /explain
2. Uma vez que a última célula executou, você deve ver um output similar a: "(1) MLflow run — Logged 1 run to an experiment in MLflow."
3. Sinta-se à vontade para clicar em **'1 run'** ou **'experiment'** para explorar o que foi criado.
4. Finalmente, teste o modelo. Digite o seguinte no Genie Code:

*Teste o modelo com uma propriedade exemplo: 2 quartos em São Francisco com 2 banheiros, para prever o base\_price.*

## Lab 7b: Predição de Preços com Machine Learning (Auto-Dirigido)

**Objetivo:** Usar o Genie Code para construir, treinar e avaliar um modelo de ML que preveja `base_price` — construa o prompt você mesmo (**consulte o Lab 7a se precisar de ajuda**).

1. **\*\*\* IMPORTANTE \*\*\*** este lab requer que o pipeline do Lab 5a ou 5b tenha completado com sucesso.
2. **'+ New' → 'Notebook'**. Linguagem padrão **'Python'**. Painel Genie Code aberto. **'New Chat'**. Modo **Agent**.
3. **Ponto de partida:** Um notebook Python vazio, com `@workspace.genie_demo.gold_price_prediction_features` como sua tabela de features.
4. **Objetivo:** Um pipeline de ML end-to-end que produza um modelo treinado para prever `base_price`, com métricas de avaliação documentadas e capacidade de pontuar uma nova propriedade descrita em linguagem natural.

5. **Restrições:** EDA antes de modelar. Split de treino/teste sem data leakage. Logar no MLflow. Comparar pelo menos dois modelos. Reportar métricas em linguagem simples. Terminar com uma predição.
  6. **Pronto quando:** Modelo treinado, métricas registradas, MLflow run visível, e predição para uma propriedade que você descreveu.
- 

## Lab 8: Analisando Texto com Funções AI SQL

**Objetivo:** Usar funções AI SQL integradas do Databricks para classificar, resumir e gerar conteúdo a partir de texto não estruturado — tudo diretamente em SQL, sem configuração de model serving ou MLflow.

**Novidade neste lab:** AI Functions são funções SQL integradas que chamam modelos de linguagem grande — sem configuração de model serving, sem MLflow, apenas SQL. Elas executam em endpoints Foundation Model API gerenciados pelo Databricks e respeitam governança do Unity Catalog. Elas contam contra sua cota diária de tokens — mantenha cláusulas LIMIT modestas enquanto estiver explorando. Estamos usando apenas 3 das funções disponíveis — veja a [documentação](#), [Enrich data using AI Functions](#), 'ai\_classify', 'ai\_summarize', 'ai\_query'.

1. **'+ New' → 'Notebook'.** Linguagem padrão **'SQL'**. Painel Genie Code aberto. **'New Chat'**. Modo **Agent**.
2. Comece com um reconhecimento rápido do dataset:

*O que a tabela @samples.bakehouse.media\_customer\_reviews contém? Mostre o schema e 5 linhas de exemplo com o texto completo do review.*

1. O Genie Code pode fazer a amostragem no chat ou no Notebook — aceite e execute a query gerada. Confirme que o conjunto de resultados tem a coluna de texto livre **'review'** — tudo o que segue a utiliza.
2. Primeiro vamos usar **ai\_classify**. Crie uma nova célula SQL com a query abaixo:

```
SELECT
  ai_classify(
    'The sourdough was incredible, crispy crust and perfect crumb. Coffee
    was average.',
    ARRAY('positive', 'negative', 'mixed', 'neutral')
  ) AS sentiment;
```

1. Execute a célula. Deve retornar **'mixed'**.
2. Aplique a função em escala. Peça ao Genie Code:

*Classifique cada linha em @samples.bakehouse.media\_customer\_reviews como positive, negative, mixed ou neutral usando ai\_classify, e retorne a contagem por label ordenada pela mais comum.*

1. Aceite e execute o SQL gerado. Você agora tem a distribuição de sentimento em todo o conjunto de reviews — sem deploy de modelo, apenas SQL.
2. Agora vamos usar **ai\_summarize**. Crie uma nova célula SQL:

```
SELECT
  ai_classify(review, ARRAY('positive', 'negative', 'mixed', 'neutral')) AS
    sentiment,
  ai_summarize(array_join(collect_list(review), ' | '), 40) AS summary
FROM samples.bakehouse.media_customer_reviews
GROUP BY 1;
```

1. Você deve obter uma linha por label de sentimento, cada uma com um resumo destilado de todos os reviews naquele grupo. O segundo argumento do **ai\_summarize** (**40**) é o comprimento máximo do resumo em palavras — mude para **15** e reexecute para ver o efeito na concisão.
2. Em seguida vamos usar **ai\_query**. Crie uma nova célula SQL:

```
SELECT ai_query('databricks-meta-llama-3-3-70b-instruct','Based on these
  bakery customer reviews, write a 2-sentence marketing tagline in British
  English. Return only the tagline, no preamble. Reviews: '
  ||array_join(collect_list(review),' ||| ')) AS tagline
FROM (
  SELECT review
  FROM samples.bakehouse.media_customer_reviews
  WHERE ai_classify(review,ARRAY('positive', 'negative', 'mixed',
    'neutral')) = 'positive'
  LIMIT 10);
```

1. Execute e inspecione o tagline. Reexecute a célula algumas vezes — o output varia porque o modelo é não-determinístico. Tente editar o prompt (ex: **"write a formal tagline"** vs **"write a playful one"**) para ver como a formulação do prompt molda o resultado.
2. Combine tudo em uma view. Peça ao Genie Code:

*Usando @samples.bakehouse.media\_customer\_reviews, crie uma view no schema workspace.genie\_demo chamada review\_insights com estas colunas: o review original, um label de sentimento do ai\_classify, e um resumo de uma frase do ai\_summarize. Depois mostre as primeiras 10 linhas.*

1. Revise o DDL antes de executar. As funções em uma view reexecutam em cada query — ok para exploração, mas em produção você faria cache dos resultados em uma tabela.
2. Finalmente, visualize o output. Peça ao Genie Code:

*Gere um gráfico de barras com as contagens de labels de sentimento da minha view review\_insights.*

1. Aceite o gráfico e execute-o. O Genie Code deve disponibilizar o gráfico como uma segunda aba no resultado da célula ao lado de **'Table'** — algo como **'Sentiment Distribution'**.

Você agora tem um caminho end-to-end de texto bruto até BI rotulado por sentimento, tudo em SQL.

## Lab 9: Genie Spaces — Converse com seus Dados

**Objetivo:** Criar um Genie Space sobre uma tabela do Unity Catalog, curá-lo com algumas definições de negócios, e ter uma conversa em linguagem natural com seus dados — a mesma experiência que você entregaria a um stakeholder não-técnico.

**Novidade neste lab:** Genie Space (cf. Genie Code — não confunda) são uma superfície diferente, voltada para negócios: uma interface de chat curada publicada sobre um conjunto de tabelas do Unity Catalog, projetada para ser usada por stakeholders não-técnicos. Você molda o comportamento do Genie Space adicionando Instructions (definições de negócios, regras, vocabulário) e Certified Queries (SQL validado que o Genie reutilizará para perguntas de alto risco em vez de gerar SQL do zero). Este lab apenas arranha a superfície do que é possível. Veja [What is a Genie Space](#), [Curate an effective Genie Space](#).

1. No menu da esquerda, clique em **'Compute'**, depois **'SQL warehouses'**, depois **'Serverless Starter Warehouse'**. Se o Status não for **'Running'**, clique **'Start'** no topo direito.
2. No menu da esquerda, clique em **'Genie'**, depois **'+ New'** para criar um novo Genie Space.
3. Em **'Connect your data'** digite **'samples.nyctaxi.trips'** — selecione a tabela da lista, depois clique **'Create'**.
4. Na caixa de chat (onde diz **'ask your question ...'**), mude de **'Agent'** para **'Chat'**.
5. Agora digite:

*Quantas viagens existem neste dataset e qual período de datas eles cobrem?*

1. Revise a resposta gerada. Clique **'Show code'** para ver o SQL que o Genie produziu — é assim que você verifica que o Genie está fazendo a coisa certa.
2. Pergunte algo analítico:

*Qual é o valor médio da corrida por hora do dia? Mostre o resultado como um gráfico de barras.*

1. Faça um follow-up na mesma conversa para mostrar contexto conversacional:

*Agora mostre a mesma coisa, mas apenas para viagens nos fins de semana.*

1. Note a forma — cada pergunta recebe uma query SQL, uma resposta, um gráfico. Isso é o modo Chat: rápido, focado, bom para perguntas conhecidas.
2. Na caixa de chat, mude de **'Chat'** para **'Agent'**:

*As corridas de fim de semana pareciam diferentes das de dia de semana. Investigue por quê. Analise a corrida média, distância da viagem, viagens por hora e distribuição de localização de embarque. Me diga quais fatores mais explicam a diferença entre fim de semana e dia de semana e onde ela aparece com mais força.*

1. Observe a stream de pensamento (clique em **'Thinking ...'**) conforme o Agent mode planeja sua abordagem, executa várias queries em paralelo, aprende de cada resultado e itera.
2. Revise o relatório final. Você deve ver um resumo estruturado com múltiplos gráficos e tabelas de apoio, e citações de volta para o SQL do Genie.
3. Expanda as citações para inspecionar qualquer query que você queira verificar.
4. Em resumo:
  - **Chat** — SQL one-shot, resultado imediato, melhor para perguntas do tipo "o que é X?"
  - **Agent** — plano + múltiplas queries + síntese, mais lento, melhor para perguntas do tipo "por que X está acontecendo?" ou "encontre padrões em Y"

1. Nas configurações do space, encontre **'Instructions'**, **'General Instructions'** e cole:

*- Uma "viagem longa" é qualquer viagem com trip\_distance maior que 10 milhas.  
- Sempre exclua viagens onde fare\_amount é 0 ou negativo — são cancelamentos.  
- Ao agrupar por localização, use pickup\_zip.*

1. Clique **'Save'**. Inicie um novo chat em modo **'Chat'** e pergunte:

*Qual proporção de todas as viagens são viagens longas, e qual pickup zip tem mais viagens longas?*

1. Confirme que o Genie aplica suas definições sem ser informado.
2. Agora faça uma pergunta que o Genie não consegue responder — igual a antes, mas 'realmente longa' em vez de 'longa':

*Qual proporção de todas as viagens são viagens realmente longas?*

1. O Genie deve responder algo como "The question cannot be answered because "really long trip" is not defined. Please clarify what trip distance should be considered a "really long trip".'
2. Forneça a definição na janela do chat:

*uma viagem realmente longa é mais de 15 milhas*

1. O Genie agora executará a query. Mas essa definição só existe neste chat — adicione-a a Instructions para disponibilizá-la a todos.
2. Finalmente, adicione uma Certified Query para que uma resposta de alto risco seja sempre consistente. Nas configurações do space, sob **'SQL Queries'**, clique **'Add'**.
3. Em **'What question does this query answer?'** digite:

*Qual é a hora mais movimentada do dia por contagem de viagens?*

1. Na caixa de texto SQL digite:

```
SELECT hour(tpcp_pickup_datetime) AS hour_of_day, count(*) AS trips
FROM samples.nyctaxi.trips
WHERE fare_amount > 0
GROUP BY 1
ORDER BY trips DESC
LIMIT 24
```

1. Clique **'Preview'** para garantir que executa e depois **'Save'**.
2. Inicie um novo chat em modo **'Chat'** e pergunte:

*Qual é a hora mais movimentada do dia por contagem de viagens?*

1. O Genie executa seu SQL certificado em vez de gerar SQL novo (veja clicando em 'Show code').

## Lab 10: Personalizar o Genie Code

**Objetivo:** Ver em primeira mão como o output do Genie Code muda quando skills são carregadas. Construir o mesmo pipeline duas vezes — uma sem orientação, outra com duas skills instaladas — e comparar.

**Novidade neste lab:** Genie Code skills são arquivos markdown curtos seguindo o padrão aberto [Agent Skills](#) — cada um é um conjunto de regras específicas de tarefa que o Genie Code aplica quando relevante. Eles residem no seu workspace Databricks em `/Users/<seu-email>/.assistant/skills/`, são descobertos automaticamente na próxima vez que você iniciar um chat, e também podem ser invocados explicitamente com `@skill-name`. Instructions são um arquivo markdown separado e único em `/Users/<seu-email>/.assistant_instructions.md` que o Genie Code lê no início de cada chat. Use-o para dizer ao Genie Code *quais* skills aplicar por padrão e *quando*. Juntos, skills + instructions permitem incorporar seus padrões corporativos no próprio Genie Code. Veja [Genie Code overview](#) e [genie-code-skills-demo on GitHub](#).

1. **'+ New' → 'ETL pipeline'**. Linguagem padrão **'SQL'**. Painel Genie Code aberto. **'New Chat'**. Modo **Agent**.
2. O primeiro passo é fazer o baseline de um Lakeflow Spark Declarative Pipeline sem skills customizadas.
3. Na UI do ETL Pipeline, mude o catálogo e schema do pipeline de **'workspace.default'** para **'workspace.genie\_demo'** (topo esquerdo).
4. Vamos começar pedindo ao Genie Code para construir um pipeline de sentimento sem orientação extra. Submeta o seguinte prompt:

*Construa um Lakeflow Spark Declarative Pipeline que leia samples.bakehouse.media\_customer\_reviews e produza um dataset de sentimento. Cada linha de saída deve incluir o texto original do review, um label*



*de sentimento geral, uma classificação de tópico e quaisquer entidades nomeadas extraídas do review.*

1. O Genie Code deve eventualmente pedir para fazer um dry-run do pipeline. Clique **'Allow'**.
2. Uma vez que completou o dry-run, clique **'Skip'**.
3. No canvas clique **'Accept'** para aceitar o código.
4. Inspeção o(s) arquivo(s) SQL gerado(s) notando o que está lá (e o que falta).  
Observações típicas do baseline:
  - Nomes de tabelas podem ou não seguir uma convenção consistente `bronze_/silver_/gold_`
  - Sem colunas de auditoria (`audit_timestamp`, `source_system`)
  - Sem tags TBLPROPERTIES de qualidade
  - Funções AI podem ser chamadas diretamente em uma única tabela bronze (mais barato chamá-las uma vez na silver)
  - Sem schema de output padronizado

Este código está ok — ele só não segue nenhum padrão. É exatamente isso que as skills vão corrigir.

1. O próximo passo é criar um Lakeflow Spark Declarative Pipeline com skills customizadas. As skills que usaremos estão no repo [genie-code-skills-demo](https://github.com/databricks-solutions/genie-code-skills-demo) no GitHub. Vamos instalar três delas no nível de usuário — sem necessidade de direitos de admin.
2. Clique em **Workspace** no menu da esquerda, depois navegue até **Workspace** → **Users** → **'<seu-email>'**.
3. Clique com botão direito na sua pasta de usuário → **Create** → **Folder**. Nomeie-a **'assistant'** (note o ponto inicial).
4. Dentro de `.assistant`, crie outra pasta chamada **'skills'**. Você agora deve ter: `/Workspace/Users/<seu-email>/.assistant/skills/`
5. Na sua pasta skills, crie três pastas nomeadas **table-governance**, **sdp-basics** e **sentiment-analysis**.
6. Na pasta `/Workspace/Users/<seu-email>/.assistant/skills/table-governance`, clique com botão direito → **Create** → **File**. Nomeie-o **'SKILL.md'**.
7. Em uma nova aba do navegador abra [https://github.com/databricks-solutions/genie-code-skills-demo/blob/main/skills/data\\_eng/table-governance.md](https://github.com/databricks-solutions/genie-code-skills-demo/blob/main/skills/data_eng/table-governance.md)
8. Faça uma leitura rápida pelo arquivo para ver como as skills são definidas.
9. Clique **Raw** (topo direito) para que o conteúdo markdown em texto plano fique visível. Copie e cole o conteúdo bruto no SKILL.md.
10. Repita para [https://github.com/databricks-solutions/genie-code-skills-demo/blob/main/skills/data\\_eng/sdp-basics.md](https://github.com/databricks-solutions/genie-code-skills-demo/blob/main/skills/data_eng/sdp-basics.md) no `/Workspace/Users/<seu-email>/.assistant/skills/sdp-basics/SKILL.md`.



11. Repita para  
<https://github.com/databricks-solutions/genie-code-skills-demo/blob/main/skills/dsml/sentiment-analysis.md>  
 no  
`/Workspace/Users/<seu-email>/.assistant/skills/sentiment-analysis`  
`/SKILL.md`.
12. Você agora deve ter três pastas, cada uma com um arquivo SKILL.md, visíveis em `/Workspace/Users/<seu-email>/.assistant/skills/`.
13. **'+' New** → **'ETL pipeline'**. Linguagem padrão **'SQL'**. Painel Genie Code aberto. **'New Chat'**. Modo **Agent**.
14. Na UI do ETL Pipeline, mude o catálogo e schema de **'workspace.default'** para **'workspace.genie\_demo'**.
15. Invoque as skills explicitamente com @ (!!! certifique-se de digitar @skill-name para localizar cada skill):

*use @table-governance, @sdp-basics e @sentiment-analysis como padrões. Construa um Lakeflow Spark Declarative Pipeline que leia samples.bakehouse.media\_customer\_reviews e produza um dataset de sentimento. Cada linha de saída deve incluir o texto original do review, um label de sentimento geral, uma classificação de tópico e quaisquer entidades nomeadas extraídas do review.*

1. O Genie Code deve eventualmente pedir para fazer um dry-run do pipeline. Clique **'Allow'**.
2. Uma vez que completou o dry-run, clique **'Skip'**.
3. No canvas clique **'Accept'** para aceitar o código.
  - Abra o(s) arquivo(s) SQL gerado(s) e compare com o output anterior. Você agora deve ver:
  - **Nomes de tabelas em camadas** — `bronze_reviews`, `silver_reviews_sentiment`, possivelmente `gold_reviews_summary`
  - **Comentários** em cada tabela descrevendo seu propósito
  - **'TBLPROPERTIES'** com pelo menos `"quality" = "bronze" | "silver" | "gold"`
  - ``current_timestamp() AS audit_timestamp`` e ``"<source>" AS source_system`` como as ÚLTIMAS colunas de cada SELECT
  - **Funções AI chamadas apenas na silver** — a bronze apenas passa o texto sem alteração
  - **Um schema de output de sentimento padronizado**
1. Se não quiser digitar @skill-name em cada prompt, você pode instalar um arquivo de instruções de usuário que diz ao Genie Code quais skills aplicar automaticamente.
2. Navegue até **Workspace** → **Users** → **'<seu-email>'**.
3. Clique com botão direito → **Create** → **File**. Nomeie-o **'assistant\_instructions.md'** (note o ponto inicial).
4. Abra a seguinte URL em uma nova aba:  
<https://github.com/databricks-solutions/genie-code-skills-demo/blob/main/instructions/.as>

sistant\_instructions.md — Clique **Raw** e cole o conteúdo bruto no .assistant\_instructions.md e salve.

5. **'+ New' → 'ETL pipeline'**. Linguagem padrão **'SQL'**. Pannel Genie Code aberto. **'New Chat'**. Modo **Agent**.
6. Na UI do ETL Pipeline, mude o catálogo e schema de **'workspace.default'** para **'workspace.genie\_demo'**.
7. Submeta o mesmo prompt de antes, mas desta vez você não invoca as skills explicitamente:

*Construa um Lakeflow Spark Declarative Pipeline que leia samples.bakehouse.media\_customer\_reviews e produza um dataset de sentimento. Cada linha de saída deve incluir o texto original do review, um label de sentimento geral, uma classificação de tópico e quaisquer entidades nomeadas extraídas do review.*

1. Como antes, clique **'Allow'** quando solicitado. O output agora deve se parecer com o pipeline anterior, mesmo que você nunca tenha mencionado as skills — o arquivo de instruções de usuário as aplicou para você.
2. Se ficar travado:
  - O Genie Code parece ignorar as skills. Certifique-se de ter clicado **'New Chat'** após instalá-las — o Genie Code lê a pasta de skills quando um chat inicia, não a cada mensagem. Também confirme que o nome da pasta é exatamente `.assistant` (com o ponto inicial) e os arquivos terminam em `.md`.
  - Pastas começando com ponto não aparecem no navegador do workspace. Alguns navegadores de workspace escondem pastas com prefixo de ponto. Elas ainda estão lá — você pode navegar digitando o caminho manualmente na barra de URL ou usando a opção **'Show hidden'** nas configurações do workspace.
  - Se ainda não conseguir descobrir, pergunte ao Genie Code por que ele não está usando suas skills.
1. Uma vez confortável com as skills, os próximos passos naturais são disponibilizá-las para todos os usuários no workspace:
  - Skills no nível do workspace — instale em `/Workspace/.assistant/skills/` para que cada usuário as receba (necessita admin do workspace)
  - Instructions no nível do workspace — `/Workspace/.assistant_workspace_instructions.md` (maior precedência, aplica-se a todos os usuários)
  - Conexão MCP (Model Context Protocol) — busque skills diretamente de um repo GitHub no início do chat, em vez de copiar arquivos para o workspace.

## E Agora?

Você conduziu o Genie Code por notebooks, ML, funções AI SQL e um Genie Space. Para continuar construindo sobre o que aprendeu, aqui estão alguns recursos que recomendamos — todos gratuitos.

### Continue Praticando no Seu Próprio Workspace

[Databricks Academy](#) — se a empresa onde você trabalha é um cliente Databricks existente, você deve ter acesso via seu e-mail corporativo através de [cursos gratuitos self-paced](#). Bons pontos de partida após este lab: *Generative AI Fundamentals*, *Get Started with Databricks for Data Analysis*, *Get Started with Databricks for Data Engineering*, e *Get Started with Databricks for Machine Learning*.

### Aprofunde-se nos Tópicos deste Lab

- [Genie Code Overview](#)
- [Use Genie Code, Genie Code for data science, Extend Genie Code with agent skills](#)
- [What is a Genie space. Curate an effective Genie space](#)
- [AI functions on Databricks, ai\\_query reference](#)

### Assista e Aprenda

- Os cursos Start Learning na Free Edition (linkados na página inicial) — Databricks Fundamentals, Intro to Data Engineering e Intro to AI/BI
- [Databricks on YouTube](#) — demos de produto, palestras de conferências e vídeos de novos recursos.

### Comunidade e Eventos

- [Databricks Community](#) — faça perguntas, compartilhe notebooks, navegue os fóruns de Genie e AI/BI.
- [Data + AI Summit](#) — conferência anual. Sessões passadas são gratuitas para assistir por streaming.